実行時観測情報と外部リスク指標を統合したソフトウェア脆弱性トリアージ手法

三宅 貴義[1,a)]　山内 利宏[2,b)]

概要：脆弱性スキャナが報告する脆弱性は増加しており，限られた人的リソースでの対処には，対象環境における重要度を考慮して対応する脆弱性を選別するトリアージが必要となる．SSVC や CISA BOD 26-04 などのトリアージ基準は，脆弱性固有の深刻度に加えて対象資産のネットワーク露出を考慮するが，コンポーネントが実際に実行されたか，脆弱関数へ実行が到達したかといった実行時の利用状況は考慮していない．そこで，本研究では，実行時の利用状況を既存のリスク指標と組み合わせて調査対象を選択する脆弱性トリアージ手法を提案し，脆弱性スキャナが報告する脆弱性情報に対し，脆弱バイナリや脆弱関数の実行，ネットワーク露出状況を付加するシステムを設計，実装した．評価の結果，実行またはネットワーク露出が観測されたパッケージに関する脆弱性は，スキャナが報告した全脆弱性のそれぞれ 11.20%以下，3.78%以下であり，実行時観測情報により調査対象を絞り込めることを示した．また，監視オーバヘッドは脆弱関数への初回到達時を除き 7 µs 未満であった．

Software Vulnerability Triage by Integrating Runtime Observations and External Risk Indicators

Takayoshi Miyake[1,a)]　Toshihiro Yamauchi[2,b)]

Abstract: Vulnerability scanners report an increasing number of vulnerabilities, making it difficult to address all of them with limited human resources. Triage criteria such as SSVC and CISA BOD 26-04 consider the network exposure of affected assets in addition to vulnerability severity, but do not consider runtime usage, such as whether a vulnerable component is executed or whether execution reaches a vulnerable function. In this study, we propose a vulnerability triage method that combines such runtime usage with existing risk indicators, and we designed and implemented a system that automatically adds information on the execution and network exposure of vulnerable binaries and functions to vulnerability reports generated by existing scanners. Evaluation across four Linux environments shows that vulnerabilities in executed packages account for at most 11.20% of all reported vulnerabilities, and those in network-exposed packages for at most 3.78%, demonstrating that runtime observations effectively narrow down the vulnerabilities requiring investigation. The monitoring overhead is less than 7 µs except upon the first reach of a vulnerable function.

1. はじめに

報告されるソフトウェア脆弱性の数は年々増加し続けており [1]，大規模言語モデル（Large Language Model, LLM）を用いた脆弱性検知がこの傾向をさらに加速させている [2]．また，ソフトウェア開発においてオープンソースソフトウェアや外部コンポーネントが多くの製品で利用されており，ソフトウェアサプライチェーン攻撃への対策が重要になっている．ソフトウェアサプライチェーン攻撃は増加しており，2025 年には 45 万件を超える悪性パッケージが発見されたことが報告されている [3]．これらのことから，製品に含まれるソフトウェアや各ソフトウェアが依

[1] 岡山大学 大学院環境生命自然科学研究科
Graduate School of Environmental, Life, Natural Science and Technology, Okayama University
[2] 岡山大学 学術研究院環境生命自然科学学域
Faculty of Environmental, Life, Natural Science and Technology, Okayama University
[a)] miyake2026@okayama-u.ac.jp
[b)] yamauchi@okayama-u.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

存するコンポーネントを把握し，製品に含まれる脆弱性に継続的に対処する必要がある．

このように脆弱性の報告数が増加する一方で，脆弱性への対処が追いついていないことが報告されている [4]．実際に，Verizon による調査 [5] では，エッジデバイスに関する 17 件の Known Exploited Vulnerabilities（KEV）について，組織が完全に対処するまでの期間の中央値が 32 日であったのに対し，大規模な悪用までの期間の中央値は 0 日であったことを報告している．このため，限られた人的リソースを最も効果的に活用して脆弱性に対処するためには，脆弱性のリスクや悪用可能性などを分析し，優先的に対処する脆弱性を順位付けすること（トリアージ）が必要である．

トリアージの基準として, Common Vulnerability Scoring System（CVSS）[6] や，Cybersecurity and Infrastructure Security Agency（CISA）の KEV Catalog [7] が用いられている．しかし，これらの基準は，脆弱性の深刻度や悪用状況を示す指標であり，評価対象の環境におけるコンポーネントの利用状況を考慮しない．このため，評価対象の環境で実際には利用されていないコンポーネントに起因する脆弱性を，調査対象として選択する可能性がある．

そこで，本研究では，脆弱なコンポーネントの利用状況を実行時監視により実行時観測情報として取得し，脆弱性スキャナの検出結果と対応付けて調査対象を選択する脆弱性トリアージ手法を提案する．提案手法では，プログラムおよび共有ライブラリのロード，実行，脆弱関数への到達，およびネットワーク露出を観測し，CVSS，Exploit Prediction Scoring System（EPSS），KEV などの外部リスク指標と組み合わせて，優先的に調査すべき脆弱性を絞り込む．この実例として，ネットワーク露出情報を用いて CISA Binding Operational Directive（BOD）26-04 [8] に基づく対処期限を算出する．

本研究の主な貢献を以下に示す．

(1) 既存トリアージ手法の多くがコンポーネントの利用状況を考慮していないことを示し，利用状況と外部リスク指標を統合したトリアージ手法を提案した．
(2) 利用状況，脆弱関数への到達，外部への露出を実行時監視により自動的に取得するシステムを設計，実装した．
(3) 複数の環境を用いた評価により，提案システムが 3 種の実行時観測情報を取得できることを確認し，絞り込み効果と監視オーバヘッドを評価した．

2. 既存のトリアージ手法と課題

2.1 脆弱性検出ツール（脆弱性スキャナ）

ソフトウェアサプライチェーン攻撃への対処には，製品に含まれるソフトウェアの構成把握が重要である．構成把握には，ソフトウェアコンポーネントとその依存関係を列挙した Software Bill of Materials（SBOM）が用いられる．SBOM を用いることで，コンポーネントと脆弱性情報を照合し，修正が必要なコンポーネントを特定できる．このため，SBOM の生成や，SBOM および対象環境に含まれるコンポーネントの脆弱性分析を支援するために，以下のようなツールが提供されている．

Trivy [9]： OS パッケージや言語ライブラリを走査し，SBOM を生成するツール．また，構成情報から脆弱性を検出し，CVE 情報やそれに関連する情報を出力．

Syft [10]： コンテナやファイルシステムからコンポーネントを走査し，SBOM を生成するツール．

Grype [11]： SBOM 中のコンポーネントを脆弱性情報と照合し，脆弱性に関する情報を出力するツール．また，脆弱性の静的なリスク情報に基づき，独自の Risk 値を出力．

OSV-Scanner [12]： SBOM や依存関係を記録したファイルなどからコンポーネントを取得し，脆弱性情報を出力するツール．また，一部の言語では静的解析により，脆弱性に関係するコードの到達可能性を判定．

しかし，これらのツールは，対象環境に含まれるコンポーネントや脆弱性を網羅的に検出することを目的としている．このため，限られた人的リソースで対処するには，検出結果から優先的に対処する脆弱性を選択するトリアージが必要である．

2.2 リスク指標に基づくトリアージ手法

脆弱性のトリアージ基準として，以下に示す脆弱性のリスク指標が利用されている．

CVSS： 脆弱性が悪用された場合の影響や，悪用に必要な条件などに基づき，脆弱性の深刻度を表す指標．

EPSS [13]： 公開済みの CVE が今後 30 日以内に悪用される確率を推定した指標．

KEV [7]： CISA により，実際に悪用が確認された脆弱性の一覧．

これらのリスク指標は，2.1 節で示したツールにおいても，検出結果を絞り込み，順位付けするための判断材料として利用されている．Trivy や OSV-Scanner は CVSS と深刻度を出力し，Grype は CVSS，EPSS，KEV などの情報に加えて，これらに基づく Risk 値を出力する．

しかし，Jacobs らは，高い深刻度を持つ脆弱性を優先的に対処する手法は，無作為な選択と同程度の効果となる場合があることを報告している [14]．また，Koscinski らは，CVSS，EPSS などの指標を同じ脆弱性に適用したところ，各方式による順位や分類の一致度が低いことを報告している [15]．これらのことから，脆弱性固有のリスク指標だけで優先度を決定することは難しく，対象環境固有の情報を考慮したトリアージ手法が必要であることが報告されている [16]．

© 2026 Information Processing Society of Japan

2.3 組織固有の資産を考慮したトリアージ手法

脆弱性のリスクは，脆弱性固有のものだけでなく，対象環境における資産の役割や露出状況によっても異なる．そこで，対象環境における脆弱性の影響や露出状況を考慮する以下のトリアージ基準や手法が提案されている．

SSVC [17]： 脆弱性の悪用状況，システムの露出状態，攻撃者にとっての有用性，および人的影響を判断項目とし，ソフトウェアを配備・運用する組織が取るべき対処行動を決定する基準．

BOD 26-04 [8]： 米国連邦政府機関を対象とする行政指令．KEV への掲載状況，資産のネットワーク露出，攻撃の自動化可能性（Automatable），悪用時の影響度（Technical Impact）から対処期限を決定することを定めている．

ACVRM [18]： ホストの構成，脆弱性スキャン結果，サービスの重要度，および組織外部への露出を統合し，対象環境の文脈を考慮して対処優先度を決定する手法．

Olswang らの研究 [19]： ネットワーク構成から生成した攻撃グラフを用い，攻撃経路を遮断する効果に基づいて優先度付けする手法を提案．

Eclipse Steady [20]： 脆弱性修正パッチから脆弱関数を特定し，静的解析と動的解析によりアプリケーションからの到達可能性を判定する手法．

VPChecker [21]： 脆弱性修正パッチから脆弱関数を特定し，バイナリを静的解析することで，脆弱関数への到達可能性を判定し，対処対象を絞り込む手法．

これらの基準や手法は，対象環境の資産の重要度，脆弱性が及ぼす影響，およびネットワーク露出などを考慮し，脆弱性固有のリスクより細粒度に対処優先度を決定する．

2.4 既存手法の課題

既存手法には以下の課題がある．

(1) **コンポーネントの利用状況を考慮していない**

2.1 節で示したように，脆弱性スキャナは，対象環境に存在する脆弱性を網羅的に検出するものの，該当コンポーネントが実際に利用されているかは判断しない．このため，実際には利用されていないコンポーネントと利用されているコンポーネントについて，関係する脆弱性が同優先度で報告されうる．利用されているコンポーネントは攻撃機会が多く，悪用される可能性が高いことから，脆弱性のトリアージにおいて，コンポーネントの利用の有無を考慮することは有効であると考えられる．しかし，既存手法では，コンポーネントの利用の有無を考慮していない．

(2) **環境固有の情報の取得が自動化されていない**

SSVC や BOD 26-04 などの基準は，判断基準を提供するだけであり，判断に必要な情報の取得方法を定めていない．また，一部の指標では，資産の重要度や組織の評価方針など，対象環境に応じた情報をあらかじめ設定する必要がある．限られた人的リソースでは，情報収集も運用上の負担となることが考えられる．

表 1 対象環境固有情報におけるトリアージへの利用状況と自動化への対応状況

手法	脆弱機能への到達	外部露出	利用状況	自動化
SSVC	–	✓	–	–
BOD 26-04	–	✓	–	–
ACVRM	–	✓	–	–
Olswang らの研究	–	✓	–	✓
Eclipse Steady	✓	–	–	✓
VPChecker	✓	–	–	✓
提案手法	✓	✓	✓	✓

✓: 対応あり，–: 対応なし

既存手法が対象環境固有情報を考慮しているか，および情報取得が自動化されているかを**表 1** に示す．表 1 に示すように，既存手法では，脆弱機能の利用または外部からの到達可能性が個別に考慮されている一方，コンポーネントの利用状況は考慮されていない．また，情報取得が自動化されていない手法が存在し，自動化されている手法も，特定の観点に限られている．これに対し，提案手法は 3 情報全てを自動取得しトリアージに利用する（3 章）．

3. 実行時観測情報と外部リスク指標を統合した脆弱性トリアージ手法

2.4 節では，コンポーネントの利用状況，脆弱機能への到達，外部ネットワークへの露出の 3 情報を対象環境から自動的に取得し，トリアージに利用する方法が求められていることを示した．これらを満たすトリアージ手法の要件を以下に示す．

（要件 1） コンポーネント（実行形式ファイル，ライブラリ）のロード，実行を取得できること．

（要件 2） 脆弱性に対応するコンポーネント内部の関数を特定し，該当関数の呼び出しの有無を取得できること．

（要件 3） 対象環境（計算機）において，外部ネットワークに露出しているコンポーネントを把握できること．

（要件 4） 脆弱性スキャナ等による脆弱性情報と，要件 1〜3 で取得した情報を対応付けられること．

（要件 5） 要件 1〜4 の情報取得と対応付けを，人手による事前設定を最小限として自動的に行えること．

（要件 6） 対象環境の実運用を妨げないこと．具体的には，カーネル改変や専用エージェントの導入を必要とせず Linux の標準機能で実現でき，監視対象プログラムへのオーバヘッドを抑えること．

（要件 7） 脆弱性情報と実行時観測情報から，外部リスク指標に対応する情報を出力でき，さらに利用者が実行時情報や外部リスク指標に基づいて，出力された脆弱性情報の絞り込みや並び替えが UI 上でできること．

本研究で設計するトリアージ手法の対処方針を示す．**（要件 1〜3）に対処**するため，環境固有情報を対象環境から自

© 2026 Information Processing Society of Japan

動的に取得する手法として実行時監視を採用する．具体的には，実行時監視を用いることで，環境固有情報を以下の実行時観測情報として取得する．

プログラムの実行　プログラムや共有ライブラリといったバイナリが，対象環境において実際に実行・ロードされたかを監視する．これにより，バイナリ単位での利用状況を取得する．

脆弱関数への到達　脆弱性の修正パッチ情報から脆弱性に対応する脆弱関数を特定し，対応バイナリに含まれる該当関数の到達を監視する．これにより，脆弱関数単位での利用状況を取得する．

ネットワークへの露出　プログラムが，外部ネットワークから到達可能なアドレスで待受けているかを監視する．また，ネットワークへ露出しているプログラムがロードしている共有ライブラリも露出していると判定する．これにより，バイナリ単位での外部ネットワークへの露出状況を取得する．

（要件 4）への対処として，脆弱性スキャナの検出単位であるパッケージに対して，実行時観測で取得した実行ファイルなどのコンポーネントや脆弱な関数を対応付ける機能を実現する．

（要件 5）への対処として，脆弱関数の特定からパッケージとの対応付けまでの一連の処理をシステムとして自動化する．利用者が事前に設定する必要がある情報は，脆弱性スキャナの実行対象などの最小限にとどめる．

（要件 6）への対処として，eBPF を利用して情報を取得する．OS 内部の情報は，システムコールや対応する関数をフックする．ユーザ空間で実行されるコンポーネントの関数については，uprobe を利用して対象関数の呼び出しを捕捉する．また，uprobe での関数呼び出しの捕捉は，1 回目の呼び出しに限定し，オーバヘッドを削減する．

（要件 7）への対処として，実行時観測情報だけで優先度を決定するのではなく，CVSS や EPSS などの外部リスク指標と統合し，利用状況と脆弱性の危険性の両面からトリアージを行うことを提案する．実行時観測は観測期間中の挙動のみを捉えるため，未観測であることは未使用を意味しない．このため，観測情報単独で優先度を決定するのではなく，未観測の脆弱性のリスクを外部リスク指標により考慮する．これにより，利用されていないコンポーネントに起因する脆弱性を調査対象として選択する可能性を低減しつつ，観測されなかった脆弱性の見落としのリスクを低減できる．また，外部リスク指標を用いたトリアージの具体例として，実行時に取得したネットワーク露出と，KEV，SSVC の Automatable および Technical Impact を組み合わせ，BOD 26-04 に基づく対処期限を算出する．具体的な算出基準については，4.4 節で説明する．また，絞り込みおよび並び替えを行う UI を実現する．UI の詳細は 4.5 節で説明する．

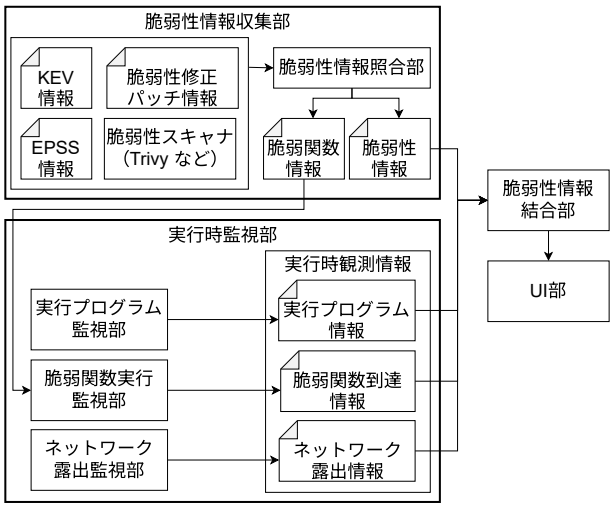


図 1　提案システムの構成とデータフロー

4. 設計

4.1 概要

提案システムの構成とデータフローを**図 1** に示し，以下に各部の概要を説明する．

脆弱性情報収集部　脆弱性スキャナの検出結果，外部リスク指標（EPSS，KEV，SSVC など）を入力とし，CVE ごとの脆弱性レコードを作成する．また，脆弱性修正パッチ情報から脆弱関数情報を抽出し，関数監視の対象として出力する．

実行時監視部　実行プログラム，脆弱関数到達，ネットワーク露出を監視する．

脆弱性情報結合部　実行時観測情報を脆弱性レコードに結合する．また，結合済みの脆弱性情報から BOD 26-04 に基づく対処期限を算出し，脆弱性レコードに付加する．

UI 部　結合済みの脆弱性情報を一覧表示する．

以降では，各部の詳細を説明する．

4.2 脆弱性情報収集部

脆弱性情報収集部は，以下の処理を行う．

(1) 脆弱性スキャナ（Trivy など）を用いて対象環境を検査することで，脆弱性を検出する．

(2) 脆弱性情報照合部は，検出された脆弱性の CVE ID を用いて，EPSS，KEV，SSVC に関する情報を照合し，脆弱性情報として出力する．

(3) 検出された脆弱性に関する修正パッチ情報から脆弱関数名を抽出し，脆弱関数情報として出力する．

4.3 実行時監視部

脆弱なパッケージの利用状況を把握するために，実行プログラム，脆弱関数到達，およびネットワーク露出を監視する．

© 2026 Information Processing Society of Japan

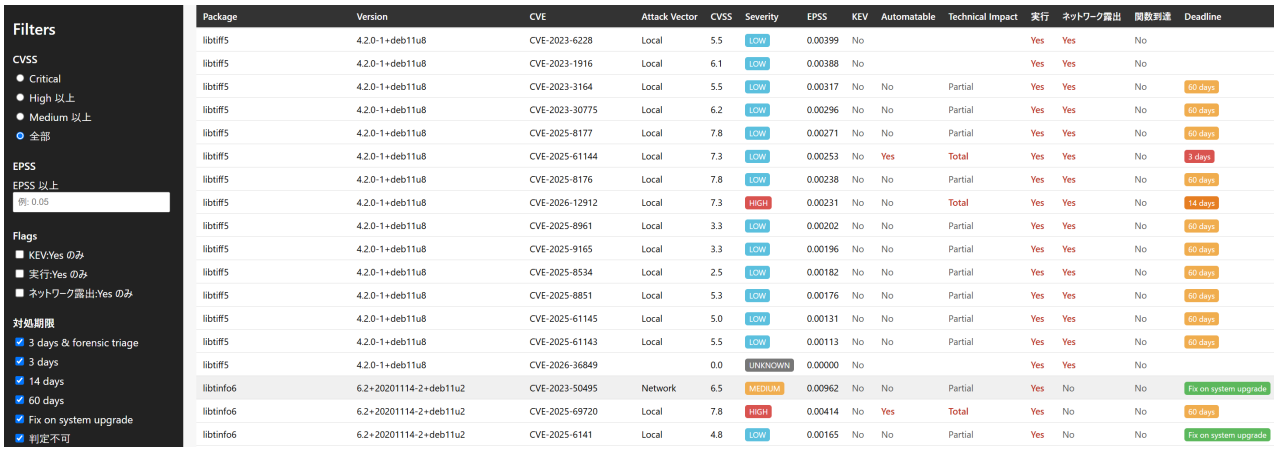


図 2　提案システムの UI 部

実行プログラム監視部：　起動時に/proc を走査し，すでに実行中のプロセスおよびロード済み共有ライブラリを取得する．さらに，eBPF により `sched_process_exec` トレースポイントと `security_mmap_file()` フックを監視し，監視開始後に実行されたプロセスと，新たにロードされた共有ライブラリを取得する．監視により取得したバイナリパスから，dpkg を用いてパッケージ名へ解決し，PID，バイナリ名，およびパッケージ名を記録する．

脆弱関数実行監視部：　脆弱関数を持つバイナリに対して，`readelf` を用いたシンボル解決を行う．シンボル解決できた関数には uprobe を設定し，対象関数の初回実行をインタラプトし，関数到達を記録する．また，インタラプトの際に，uprobe を detach し，以降は同関数の監視を行わないことで，オーバヘッドを低減する．なお，シンボルを解決できない関数は観測不能として記録する．

ネットワーク露出監視部：　起動時に/proc/net を走査し，すでに待受けを行っているプロセスと対応アドレスを取得する．さらに，eBPF を用いて `inet_listen()` および `inet_bind()` を監視し，監視開始後の待受けを監視する．監視により取得したバイナリパスから，dpkg を用いてパッケージ名へ解決し，PID，バイナリ名，およびパッケージ名を記録する．また，“127.0.0.1” または “::1” である場合をループバック，それ以外を外部公開として区別して記録する．

4.4　脆弱性情報結合部

脆弱性情報収集部が生成した脆弱性情報に，実行時監視部が出力した実行時観測情報を付与する．結合方法は以下の通りである．

- **実行プログラム情報：**パッケージ名を用いて脆弱性レコードと結合し，脆弱性を含む実行ファイルまたは共有ライブラリの利用が観測されたレコードを実行済みとする．
- **脆弱関数到達情報：**到達を観測した関数に紐づく CVE ID とパッケージ名を用いて脆弱性レコードと結合し，対応するレコードを関数到達ありとする．
- **ネットワーク露出情報：**実行プログラム監視部の情報を用いて，外部公開プロセスと，そのプロセスがロードしていた共有ライブラリを，PID で結合する．外部公開プロセスに紐づくパッケージのレコードをネットワーク露出ありとして記録する．なお，ループバックはネットワーク露出なしとして記録する．

上記の処理により，結合部は各脆弱性レコードに，実行プログラム，関数到達，ネットワーク露出を付与する．また，既存基準への適用例として，結合済みのネットワーク露出と，KEV，SSVC の Automatable および Technical Impact を用いて，BOD 26-04 に基づく対処期限を算出する．判定は露出・KEV 登録・攻撃自動化可能性・技術的影響の 4 条件の組み合わせにより，最短 3 日から次期システム更新時までの対処期限を割り当てるものであり，判定条件の詳細は**表 A·1**（付録 A.1）に示す．なお，算出に必要な情報が不足する場合は，対処期限を表示しない．

4.5　UI 部

UI 部では，各脆弱性について，パッケージ名，バージョン，CVE，CVSS，EPSS，KEV，Automatable，Technical Impact，実行済み，ネットワーク露出，関数到達，対処期限を列として一覧表示する．実際の画面を**図 2** に示す．各カラムを選択することで，対応する値の昇順または降順に並び替えができ，利用者は，CVSS，EPSS，KEV，実行済み，ネットワーク露出，対処期限で脆弱性を絞り込める．

4.6　制約

提案手法には，以下の制約がある．

(1) パッケージ単位の結合のため，同一パッケージ内の脆弱性と無関係な実行ファイルまたは共有ライブラリが利用された場合も，そのパッケージに属する全脆弱性が実行済みとなる．

(2) 脆弱関数のパッチ情報が得られない場合，またはシン

© 2026 Information Processing Society of Japan

ボル情報が strip されている場合などは，脆弱関数監視ができない．

(3) 実行時観測は観測期間中の挙動に限られるため，定期実行ジョブなど観測期間外にのみ利用されるコンポーネントは未観測となる．このため，3 章で述べた通り，観測情報は外部リスク指標と併用する．

5. 評価

5.1 評価目的

提案手法では，実行時監視により取得した実行時観測情報を用いて，脆弱性スキャナの検出結果を絞り込むことができる．一方で，実行時監視は，対象環境の実行状態を監視するため，監視対象のプロセスにオーバヘッドを与える可能性がある．そこで，本章では，以下の 3 つの観点から提案システムを評価する．

(1) 実際のソフトウェアに関する実行時観測情報を取得できるか
(2) 観測情報によって調査対象をどの程度絞り込めるか
(3) 実行時監視のオーバヘッドはどの程度か

なお，観点 (2) の評価は，脆弱性情報と観測情報の対応付け（要件 4）およびその自動化（要件 5）が機能することを前提としており，これらも併せて確認する．

5.2 実行時観測機能の評価

5.2.1 評価方法

実行プログラムおよびネットワーク露出の観測方法：実際の環境において，実行時観測情報を取得できるかを確認するため，評価環境として，Debian 11 上で監視部を動作させた．評価環境は，2026 年 7 月 24 日時点で最新のパッケージ状態に更新し，動作確認のために追加で以下のパッケージをインストールした．

(1) Nginx: Web サーバ．“0.0.0.0” で待受け．
(2) OpenSSH: リモートシェルサーバ．“0.0.0.0” で待受け．
(3) PostgreSQL: データベースサーバ．“127.0.0.1” で待受け．
(4) Apache2: Web サーバ．起動しないよう設定．

また，上記の環境において，監視部を 5 分間動作させ，動作中に以下の作業を行った．

(1) Nginx に対して，`curl` コマンドで通信．
(2) OpenSSH に対して，`ssh` コマンドでログイン．
(3) PostgreSQL に対して，`psql` コマンドで通信．

脆弱関数到達の観測方法：実際のバイナリに対する脆弱関数実行監視部の動作を確認するため，前述の評価環境上において，Nginx を対象とした観測を行った．監視は，CVE-2023-44487 に関係する関数として，HTTP/2 通信においてヘッダの処理を行う `ngx_http_v2_state_headers` 関数を対象とした．また，HTTP/2 通信でのみ到達することを確認するために，以下の条件に分け，観測を行った．

表 2 ソフトウェアの利用状況と観測結果

ソフトウェア	利用状況	実行（観測）	露出（観測）
Nginx	外部待受	✓	✓
OpenSSH	外部待受	✓	✓
PostgreSQL	ループバック待受	✓	–
Apache2	未起動	–	–

✓: 観測された，–: 観測されなかった

表 3 Nginx への通信条件と脆弱関数到達の観測結果

条件	関数到達
通信なし	–
HTTP/1.1 通信	–
HTTP/2 通信	✓

✓: 観測された，–: 観測されなかった

(1) 監視中に Nginx に対して通信を行わない場合
(2) `curl` コマンドを用いて HTTP/1.1 通信を行う場合
(3) `curl` コマンドを用いて HTTP/2 通信を行う場合

なお，Nginx の実行バイナリはシンボル情報が strip されているため，Debian により提供されている分離デバッグ情報を用いて観測対象関数のアドレスを解決した．

5.2.2 評価結果

実行プログラムおよびネットワーク露出の観測結果：各ソフトウェアの利用状況と観測結果を**表 2** に示す．表 2 に示す通り，各ソフトウェアについて，設定した実行状態およびネットワーク露出（外部待受，ループバック待受，未起動）と一致する観測結果が得られた．この結果から，実行状態およびネットワーク露出を区別して観測できることを確認した．

脆弱関数到達の観測結果：Nginx への通信条件ごとの脆弱関数到達の観測結果を**表 3** に示す．表 3 に示す通り，HTTP/1.1 による通信では対象関数への到達は観測されず，HTTP/2 による通信では到達が観測された．この結果から，Nginx が実行されていることだけでなく，対象脆弱性に関係する HTTP/2 処理が利用されたことを関数単位で識別できることを確認した．

5.3 絞り込み効果の評価

5.3.1 評価方法

提案システムにより，どの程度の脆弱性が実行済み，ネットワーク露出済みとして判定されるかを評価した．前節で示した評価環境を含む 4 環境（Debian 11，Debian 13，Ubuntu 22.04，Ubuntu 26.04）について，それぞれ 2026 年 7 月 24 日時点で最新のパッケージ状態に更新し，脆弱性スキャナには Trivy（v0.72.0）を用いて脆弱性情報を収集した．また，観測したネットワーク露出情報と，KEV および SSVC を用いて，BOD 26-04 に基づく対処期限を算出した．

なお，脆弱関数到達の観測機能は 5.2 節で確認したが，全ての環境に共通して監視可能な脆弱関数を設定すること

© 2026 Information Processing Society of Japan

表 4 実行時観測情報による CVE の絞り込み結果

		実行		露出	
環境	全件	件数	割合（%）	件数	割合（%）
Debian 11	8,311	931	11.20	314	3.78
Debian 13	7,081	679	9.59	168	2.37
Ubuntu 22.04	52,733	125	0.24	55	0.10
Ubuntu 26.04	3,484	168	4.82	46	1.32

割合は全件に対する比率を示す

表 5 BOD 26-04 に基づく対処期限の算出結果

環境	3 日	14 日	60 日	更新時	判定不可
Debian 11	13	89	669	4,014	3,526
Debian 13	6	53	735	2,523	3,764
Ubuntu 22.04	0	63	175	14,335	38,160
Ubuntu 26.04	1	8	72	427	2,976

が困難であるため，本評価では関数到達による絞り込み効果の評価は行わなかった．

5.3.2 評価結果

実行時観測情報による絞り込み結果：実行済みおよびネットワーク露出済みのパッケージに関する脆弱性レコードの件数と割合を**表 4** に示す．なお，評価環境に含まれるパッケージや脆弱性数を付録 A.2.1，外部リスク指標による内訳を付録 A.2.2 に示す．表 4 に示す通り，実行済みのパッケージに関する脆弱性は全件の 11.20%以下，ネットワークに露出したパッケージに関する脆弱性は 3.78%以下であり，環境によっては 1%未満（Ubuntu 22.04 で 0.24%，0.10%）であった．これらの結果から，本評価の環境では，実行時観測情報を用いて脆弱性スキャナが報告した脆弱性から，実行またはネットワーク露出が観測されたパッケージに関する脆弱性を抽出できた．

対処期限の算出結果：各環境における BOD 26-04 の対処期限の算出結果を**表 5** に示し，以下に詳細を説明する．

(1) 外部露出の有無を利用し，BOD 26-04 の判定規則に従って対処期限を算出できた．ただし，SSVC が欠落している CVE では BOD 26-04 に基づく期限を算出できず，本評価では最大で約 85%の脆弱性が判定不可に分類された．また，全ての環境において，forensic triage を伴う対処に分類されたレコードは確認されなかった．

(2) Debian 11 において，3 日以内に対処が必要な脆弱性 13 件のうち 7 件は Nginx に関する KEV の脆弱性であり，ネットワーク露出が観測されたことから，このように分類された．一方で，Debian 13 において，1 件のみ確認された KEV の脆弱性は，ネットワーク露出が観測されなかったことから，14 日以内での対処と分類された．これらのことから，提案システムは，悪用可能性に関する情報だけでなく，対象環境におけるネットワーク露出を反映して，BOD 26-04 に基づく対処期限を自動的に算出できた．

表 6 各監視条件における評価用プログラムの実行時間とオーバヘッド

条件	実行時間（µs）	オーバヘッド（µs）	オーバヘッド（%）
監視なし	863.1 ± 154.2	–	–
実行監視	868.9 ± 154.7	5.8	0.7
露出監視	863.5 ± 151.1	0.4	0.05
関数監視（初到達）	$1{,}410.2 \pm 196.3$	547.1	63.4
関数監視（到達後）	865.4 ± 149.9	2.3	0.3
全監視（初到達）	$1{,}420.2 \pm 210.0$	557.1	64.5
全監視（到達後）	869.7 ± 156.3	6.6	0.8

実行時間は平均 ± 標準偏差（$n = 10{,}000$）を示す

5.4 監視オーバヘッドの評価

5.4.1 評価方法

各監視部がプログラム実行時間に与える影響を調べるため，以下の処理を行う評価用プログラムに対し，提案システムの監視部を有効化，または無効化した状態での実行時間を評価した．

(1) TCP ソケットを作成し，`bind()` および `listen()` を実行する．

(2) 脆弱関数実行監視部の観測対象となる関数を呼び出す．

(3) ソケットを閉じて終了する．

また，監視部の有効化，無効化について以下の組み合わせで評価した．

- 監視なし: 全監視部を無効化
- 実行監視: 実行プログラム監視部のみ有効化
- 露出監視: ネットワーク露出監視部のみ有効化
- 関数監視（初到達）: 脆弱関数実行監視部のみ有効化し，対象関数への初回到達時の実行時間を測定
- 関数監視（到達後）: 脆弱関数実行監視部のみ有効化し，対象関数への到達確認後の実行時間を測定
- 全監視（初到達）: 全監視部を有効化し，対象関数への初回到達時の実行時間を測定
- 全監視（到達後）: 全監視部を有効化し，対象関数への到達確認後の実行時間を測定

上記の 7 条件それぞれについて，評価用プログラムを 10,000 回実行し，実行に要した実時間の平均値，標準偏差を取得した．評価は，Ubuntu 26.04 LTS（kernel 7.0.0-29-generic），AMD Ryzen 7 PRO 6850U，32GB メモリ上で行った．

5.4.2 評価結果

評価結果を**表 6** に示し，以下に詳細を述べる．

(1) 脆弱関数実行監視部による初到達の場合を除き，監視部によるオーバヘッドは 7 µs 未満であった．

(2) 脆弱関数実行監視部による初到達の場合，547.1 µs のオーバヘッドが発生した．これは，初回到達時に uprobe によるインタラプト処理が発生するためであり，初回到達後は 2.3 µs まで低減した．

© 2026 Information Processing Society of Japan

5.5 考察

(1) 実際のソフトウェアに関する実行時観測情報を取得できるか

4 つのソフトウェアについて，設定した実行状態およびネットワーク露出と一致する情報を取得できた．また，Nginx を対象とした観測では，HTTP/2 処理に起因する脆弱性について，HTTP/2 通信でのみ関数到達の観測を確認した．

(2) 観測情報によって調査対象をどの程度絞り込めるか

実行済みおよびネットワーク露出済みのパッケージに関する脆弱性は，最大でも 11%程度であり，環境によっては 1%未満にまで絞り込めていた．また，BOD 26-04 に基づく対処期限の算出結果から，提案システムは，悪用可能性に関する情報だけでなく，対象環境におけるネットワーク露出を反映して対処期限を自動的に算出できた．

(3) 実行時監視のオーバヘッドはどの程度か

実行時監視のオーバヘッドは，脆弱関数実行監視部による初回到達時のオーバヘッドが最大の 557.1 µs であった．しかし，1 回でも関数到達を観測すれば，その後のオーバヘッドは 2.3 µs まで低減することに加え，監視対象外のソフトウェアは実行時間に対する影響を受けない．このため，提案システムのオーバヘッドは多くの場合で 7 µs 未満である．

6. おわりに

既存の脆弱性対処指標が対象環境における脆弱コンポーネントの利用状況を考慮していないことを明らかにし，プログラムの実行，脆弱関数到達，およびネットワーク露出の実行時観測情報を，実運用環境から自動取得して脆弱性と対応付けるシステムを設計，実装した．さらに，実行時観測情報と外部リスク指標を統合することで，未観測の脆弱性のリスクも考慮するトリアージ手法を示した．

評価の結果，実行が観測されたパッケージに関係する脆弱性はスキャナの報告全体の 11.20%以下，ネットワーク露出が観測されたものは 3.78%以下であり，実行時観測情報を調査対象の絞り込みに利用できることを示した．また，監視オーバヘッドは脆弱関数への初回到達時を除き 7 µs 未満であり，実運用環境への適用が可能であることを示した．

謝辞 本研究の一部は，JST 経済安全保障重要技術育成プログラム JPMJKP24K2 の支援を受けたものです．

参考文献

[1] Elder, S., Rahman, M. R., Fringer, G. et al.: A Survey on Software Vulnerability Exploitability Assessment, *ACM Comput. Surv.*, Vol. 56, No. 8, pp. 1–41 (2024).

[2] Gamblin, J. and Leverett, É.: The 2026 Vulnerability Forecast Update: Navigating the AI Epoch, FIRST (online), available from ⟨https://www.first.org/blog/20260615-vulnerability-forecast-update⟩ (accessed 2026-08-06).

[3] Sonatype: 2026 State of the Software Supply Chain, Technical report, Sonatype (2026).

[4] Veracode: 2026 State of Software Security: Prioritize, Protect, Prove, Technical report, Veracode (2026).

[5] Verizon Business: 2025 Data Breach Investigations Report, Technical report, Verizon Business (2025).

[6] FIRST: Common Vulnerability Scoring System (CVSS), FIRST (online), available from ⟨https://www.first.org/cvss/⟩ (accessed 2026-08-06).

[7] CISA: Known Exploited Vulnerabilities Catalog, CISA (online), available from ⟨https://www.cisa.gov/known-exploited-vulnerabilities-catalog⟩ (accessed 2026-06-30).

[8] CISA: BOD 26-04: Prioritizing Security Updates Based on Risk, CISA (online), available from ⟨https://www.cisa.gov/news-events/directives/bod-26-04-prioritizing-security-updates-based-risk⟩ (accessed 2026-06-30).

[9] Aqua Security: Trivy, GitHub (online), available from ⟨https://github.com/aquasecurity/trivy⟩ (accessed 2026-06-03).

[10] Anchore: Syft, GitHub (online), available from ⟨https://github.com/anchore/syft⟩ (accessed 2026-08-20).

[11] Anchore: Grype, GitHub (online), available from ⟨https://github.com/anchore/grype⟩ (accessed 2026-08-20).

[12] Google: OSV-Scanner, Google LLC (online), available from ⟨https://google.github.io/osv-scanner/⟩ (accessed 2026-08-20).

[13] FIRST: Exploit Prediction Scoring System (EPSS), FIRST (online), available from ⟨https://www.first.org/epss/⟩ (accessed 2026-06-30).

[14] Jacobs, J., Romanosky, S., Adjerid, I. et al.: Improving Vulnerability Remediation through Better Exploit Prediction, *J. Cybersecur.*, Vol. 6, No. 1, p. tyaa015 (2020).

[15] Koscinski, V., Nelson, M., Okutan, A. et al.: Conflicting Scores, Confusing Signals: An Empirical Study of Vulnerability Scoring Systems, *Proc. ACM CCS 2025*, pp. 1904–1918 (2025).

[16] Jiang, Y., Oo, N., Meng, Q. et al.: A Survey on Vulnerability Prioritization: Taxonomy, Metrics, and Research Challenges, arXiv:2502.11070 (online), available from ⟨https://arxiv.org/abs/2502.11070⟩ (accessed 2026-08-18).

[17] Spring, J., Householder, A., Hatleback, E. et al.: Prioritizing Vulnerability Response: A Stakeholder-Specific Vulnerability Categorization (Version 2.0), Technical report, Software Engineering Institute, Carnegie Mellon University (2021).

[18] Mehri, V. A., Arlos, P. and Casalicchio, E.: Automated Context-Aware Vulnerability Risk Management for Patch Prioritization, *Electronics*, Vol. 11, No. 21, p. 3580 (2022).

[19] Olswang, A., Gonda, T., Puzis, R. et al.: Prioritizing Vulnerability Patches in Large Networks, *Expert Syst. Appl.*, Vol. 193, p. 116467 (2022).

[20] Ponta, S. E., Plate, H. and Sabetta, A.: Detection, Assessment and Mitigation of Vulnerabilities in Open Source Dependencies, *Empir. Softw. Eng.*, Vol. 25, pp. 3175–3215 (2020).

[21] Patel, H., Snit, A. and Polychronakis, M.: Supply Chain Reaction: Enhancing the Precision of Vulnerability Triage Using Code Reachability Information, *Proc. ACSAC 2025*, pp. 92–107 (2025).

© 2026 Information Processing Society of Japan

表 A·1 BOD 26-04 に基づく対処期限の判定

			対処期限	
露出	KEV	自動化 *1	影響 *2: Total	Partial
✓	✓	✓	3 days*3	3 days
✓	✓	–	3 days*3	14 days
✓	–	✓	3 days	14 days
✓	–	–	14 days	60 days
–	✓	✓	3 days*3	14 days
–	✓	–	14 days	14 days
–	–	✓	60 days	60 days
–	–	–	Upgrade*4	Upgrade*4

*1 Automatable: 攻撃者が攻撃を自動化できるかどうか.
*2 Technical Impact: Total は完全なシステムの奪取，Partial は部分的な影響を意味する.
*3 forensic triage（すでに攻撃が行われているかの検査）を併せて実施する.
*4 次期システム更新時に修正する（Fix on system upgrade）.

表 A·2 評価対象のパッケージおよび脆弱性の規模

	パッケージ数		脆弱性数	
環境	総数	脆弱性関連	レコード	CVE
Debian 11	1,784	556	8,311	3,131
Debian 13	1,746	554	7,081	1,756
Ubuntu 22.04	1,592	214	52,733	6,513
Ubuntu 26.04	1,548	172	3,484	491

付　　録

A.1　BOD 26-04 に基づく対処期限の判定条件

4.4 節で述べた BOD 26-04 に基づく対処期限の判定条件の詳細を表 A·1 に示す．対処期限は，ネットワーク露出の有無，KEV への登録有無，SSVC の Automatable（攻撃者が攻撃を自動化できるか）および Technical Impact（脆弱性が及ぼす影響．Total は完全なシステムの奪取，Partial は部分的な影響を意味する）の 4 条件の組み合わせにより決定される．露出ありかつ KEV 登録済みの脆弱性には最短 3 日の期限と forensic triage（すでに攻撃が行われているかの検査）が課され，いずれの条件にも該当しない脆弱性は次期システム更新時の修正とされる．

A.2　絞り込み評価結果の詳細

A.2.1　評価環境の詳細

本評価では，1 つの CVE が複数のパッケージに影響する場合があるため，脆弱性レコードを CVE ID とパッケージ名の組み合わせで定義した．このため，CVE ID が同一であっても，パッケージ名が異なる場合は別の脆弱性レコードとして扱っている．各環境におけるパッケージ数および脆弱性レコード数を**表 A·2** に示す．なお，脆弱性レコードには，CVE ID 以外にディストリビューション固有の一時的な脆弱性識別子を持つレコードも含まれる．

表 A·3 各外部リスク指標に該当した脆弱性レコード数

項目	D11	D13	U22	U26
全レコード	8,311	7,081	52,733	3,484
CVSS $9.0 \leq x \leq 10.0$	188	114	66	29
CVSS $7.0 \leq x < 9.0$	2,584	1,922	14,184	1,281
CVSS $4.0 \leq x < 7.0$	4,845	4,262	37,954	2,057
CVSS $0.0 \leq x < 4.0$	694	783	529	117
EPSS $0.9 \leq x \leq 1.0$	9	0	18	0
EPSS $0.5 \leq x < 0.9$	44	47	0	0
EPSS $0.1 \leq x < 0.5$	42	25	20	0
EPSS $0.01 \leq x < 0.1$	1,011	551	399	193
EPSS $0.0 \leq x < 0.01$	7,205	6,458	52,296	3,291
SSVC 取得あり	4,785	3,317	14,573	508
SSVC 欠落	3,526	3,764	38,160	2,976
KEV	9	1	42	0
Automatable: yes	604	700	164	52
Automatable: no	4,181	2,617	14,409	456
Technical Impact: total	814	604	2,267	145
Technical Impact: partial	3,971	2,713	12,306	363

D11: Debian 11, D13: Debian 13
U22: Ubuntu 22.04, U26: Ubuntu 26.04

A.2.2　外部リスク指標の内訳

外部リスク指標に該当した脆弱性レコード数を**表 A·3** に示す．表 A·3 では，全レコードに対し，各外部リスク指標を満たすレコード数を示している．CVSS と EPSS については，階級分けを行ったうえで，それぞれの値域に該当するレコード数を示している．

© 2026 Information Processing Society of Japan